**Register Number:** 192425207

**Name:** Anand Paul . D

**CO1 – AT1**

**MLA0201 – Fundamentals of Machine Learning**

**Application- Based Questions**

1. Implement the concept of probability distribution to predict the likelihood of spam emails in a mail free trail system. give me the python code to implement in python idle and to get the output.

## Code:

spam_probability.py

C: > Users > Anandpaul > AppData > Local > Programs > Python > Python314 > spam_probability.py

```python
import math
from collections import defaultdict

# ------------------------------------------------------------
# 1. TRAINING DATASET (Simulated Free Trial System)
# ------------------------------------------------------------
training_emails = [
    ("claim your free trial extension now win prize", "spam"),
    ("free trial account registration confirmed welcome", "ham"),
    ("urgent click here to get free bonus credit card required", "spam"),
    ("your free trial account details and login info", "ham"),
    ("congratulations you win cash money free trial now", "spam"),
    ("meeting scheduled regarding project update tomorrow", "ham"),
    ("verify your email address for free trial access", "ham"),
    ("click link to claim free gift card money instantly", "spam")
]

# ------------------------------------------------------------
# 2. PROBABILITY DISTRIBUTION TRAINER
# ------------------------------------------------------------
class NaiveBayesSpamFilter:
    def __init__(self, alpha=1.0):
        # alpha = 1.0 applies Laplace Smoothing (prevents zero probabilities)
        self.alpha = alpha
        self.prior_spam = 0.0
        self.prior_ham = 0.0
        self.word_counts_spam = defaultdict(int)
        self.word_counts_ham = defaultdict(int)
        self.total_words_spam = 0
        self.total_words_ham = 0
        self.vocab = set()

    def train(self, dataset):
        total_emails = len(dataset)
        spam_count = sum(1 for _, label in dataset if label == "spam")
        ham_count = total_emails - spam_count

        # Prior Probabilities P(Spam) & P(Ham)
        self.prior_spam = spam_count / total_emails
        self.prior_ham = ham_count / total_emails

        # Counting word frequencies
        for text, label in dataset:
            words = text.lower().split()
            for word in words:
                self.vocab.add(word)
                if label == "spam":
                    self.word_counts_spam[word] += 1
                    self.total_words_spam += 1
                else:
                    self.word_counts_ham[word] += 1
                    self.total_words_ham += 1
```

## Output:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

= RESTART: C:/Users/Anandpaul/AppData/Local/Programs/Python/Python314/spam_probability.py
============================================================
SPAM FILTERING PROBABILITY DISTRIBUTION ENGINE
============================================================
Prior Probability P(Spam) = 50.0%
Prior Probability P(Ham)  = 50.0%

Email Text: "Claim your free bonus money now"
  ├─ Likelihood Spam :  96.24%
  ├─ Likelihood Ham  :   3.76%
  └─ Prediction      : [SPAM]

Email Text: "Your free trial account password reset request"
  ├─ Likelihood Spam :  12.51%
  ├─ Likelihood Ham  :  87.49%
  └─ Prediction      : [HAM (Legitimate)]

Email Text: "Urgent click link to win cash prize"
  ├─ Likelihood Spam :  99.55%
  ├─ Likelihood Ham  :   0.45%
  └─ Prediction      : [SPAM]

Email Text: "Welcome to your trial project setup meeting"
  ├─ Likelihood Spam :   9.64%
  ├─ Likelihood Ham  :  90.36%
  └─ Prediction      : [HAM (Legitimate)]
```

2. **Develop the design steps of a learning system to build a machine learning solution for student performance prediction. for this qstn too code**

## Code:

spam_probability.py | student_performance.py

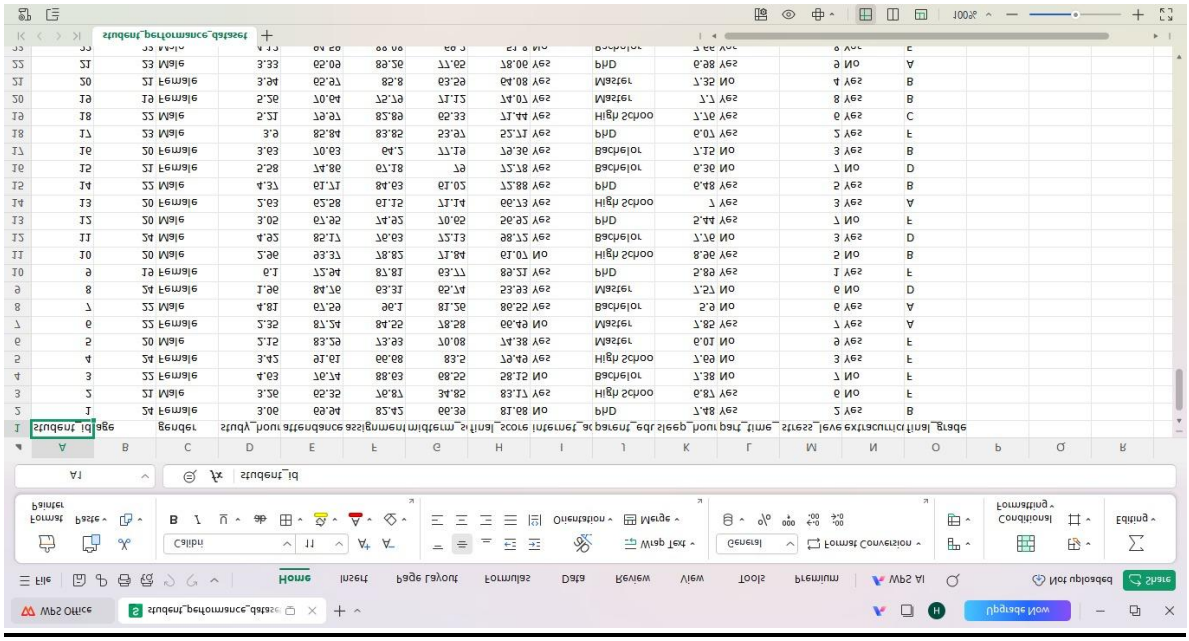
: > Users > Anandpaul > AppData > Local > Programs > Python > Python314 > student_performance.py > ...

```python
import math

# ---------------------------------------------------------------
# 1. HISTORICAL TRAINING DATASET
# Features: [Study Hours/Week, Attendance %, Past Exam Score %]
# Target: Final Grade %
# ---------------------------------------------------------------
dataset = [
    ([10, 85, 70], 73),
    ([15, 95, 88], 91),
    ([3,  60, 50], 45),
    ([8,  78, 65], 67),
    ([12, 90, 82], 85),
    ([5,  70, 58], 54),
    ([18, 98, 92], 96),
    ([2,  50, 45], 40),
    ([14, 88, 80], 83),
    ([7,  75, 62], 62)
]

# ---------------------------------------------------------------
# 2. DATA PREPROCESSING (Feature Scaling)
# ---------------------------------------------------------------
class FeatureScaler:
    """Min-Max Normalization to scale features between 0 and 1."""
    def __init__(self):
        self.min_vals = []
        self.max_vals = []

    def fit_transform(self, X):
        num_features = len(X[0])
        self.min_vals = [min(row[j] for row in X) for j in range(num_features)]
        self.max_vals = [max(row[j] for row in X) for j in range(num_features)]

        return [self.transform_row(row) for row in X]

    def transform_row(self, row):
        scaled = []
        for j in range(len(row)):
            spread = self.max_vals[j] - self.min_vals[j]
            scaled_val = (row[j] - self.min_vals[j]) / spread if spread != 0 else 0
            scaled.append(scaled_val)
        return scaled

# ---------------------------------------------------------------
# 3. MACHINE LEARNING ENGINE (Gradient Descent Regression)
# ---------------------------------------------------------------
class StudentPerformancePredictor:
    def __init__(self, num_features, learning_rate=0.1, epochs=2000):
        self.weights = [0.0] * num_features
        self.bias = 0.0
        self.lr = learning_rate
        self.epochs = epochs

```

## Output:

```
================================================================================ RESTA
================================================================
      STUDENT PERFORMANCE PREDICTION LEARNING SYSTEM
================================================================
Model Training Status: Complete (2000 epochs)
Performance Measure (MSE) : 1.50
Performance Measure (R²)  : 99.56%

PREDICTIONS FOR NEW STUDENTS:
----------------------------------------------------------------
Student: Alice
   ├─ Metrics    : Study=16hrs/wk | Attendance=92% | Past Score=85%
   ├─ Pred Grade : 89.3%
   └─ Status     : [PASS]

Student: Bob
   ├─ Metrics    : Study=4hrs/wk | Attendance=65% | Past Score=52%
   ├─ Pred Grade : 50.2%
   └─ Status     : [PASS]

Student: Charlie
   ├─ Metrics    : Study=11hrs/wk | Attendance=80% | Past Score=75%
   ├─ Pred Grade : 74.4%
   └─ Status     : [PASS]


================================================================================ RESTA
```

## Dataset:



3. **Model the weather forcasting system with its otcome and challenges using random varaibles and probability theory.**

## Code:

C: > Users > Anandpaul > AppData > Local > Programs > Python > Python314 > weather_forecasting.py > ...

```python
import random
import math

# ==========================================================================
# 1. MARKOV CHAIN MODEL (Discrete Weather State Transitions)
# ==========================================================================
class WeatherMarkovChain:
    def __init__(self):
        self.states = ["Sunny", "Cloudy", "Rainy"]
        # Transition Probability Matrix P[i][j]: P(State_t+1 = j | State_t = i)
        # Rows: [From Sunny, From Cloudy, From Rainy]
        self.transition_matrix = [
            [0.60, 0.30, 0.10],  # From Sunny
            [0.25, 0.45, 0.30],  # From Cloudy
            [0.15, 0.35, 0.50]   # From Rainy
        ]

    def predict_future_distribution(self, current_state, days):
        """Calculates exact categorical probability distribution for N days ahead."""
        # Initial probability vector (100% for the current state)
        state_idx = self.states.index(current_state)
        curr_dist = [0.0, 0.0, 0.0]
        curr_dist[state_idx] = 1.0

        # Matrix multiplication across forecast horizon
        for _ in range(days):
            next_dist = [0.0, 0.0, 0.0]
            for i in range(3):
                for j in range(3):
                    next_dist[j] += curr_dist[i] * self.transition_matrix[i][j]
            curr_dist = next_dist

        return dict(zip(self.states, curr_dist))


# ==========================================================================
# 2. MONTE CARLO ENSEMBLE FORECAST (Continuous Variables & Chaos)
# ==========================================================================
class EnsembleWeatherForecast:
    def __init__(self, init_temp=28.0, measurement_error=0.5):
        self.init_temp = init_temp
        self.measurement_error = measurement_error
        self.chaos_factor = 0.20  # Exponential divergence rate (Lyapunov exponent)

    def run_simulation(self, days_ahead=5, num_runs=10000):
        """Runs N parallel atmospheric trajectories to model chaotic temperature outcomes."""
        outcomes = []

        for _ in range(num_runs):
            # Sample initial state from Gaussian distribution with measurement noise
            temp = random.gauss(self.init_temp, self.measurement_error)

            for day in range(1, days_ahead + 1):
                # Daily atmospheric trend + expanding uncertainty
```

**Output:**

```
================================================================
================================================================
      PROBABILISTIC WEATHER FORECASTING SYSTEM
================================================================

[PART 1] DISCRETE MARKOV CHAIN FORECAST
Current State: Sunny | Forecast Horizon: 3 Days

Predicted Probability Distribution:
   ├─ P(State = Sunny ) =  38.75%
   ├─ P(State = Cloudy) =  36.25%
   ├─ P(State = Rainy ) =  25.00%

[PART 2] MONTE CARLO ENSEMBLE TEMPERATURE FORECAST
Initial Observed Temp : 28.00°C
Expected Temp (Mean)  : 27.10°C
95% Confidence Bounds : [24.34°C  to  29.86°C]
Probability of Heatwave (>30°C) : 2.09%
================================================================
```

4. **Apply conditional probability concepts to design an fraud detection model for suspecious banking transcation. give me python code**

## Code:

spam_probability.py | student_performance.py | weather_forecasting.py | Bank_transcation.py

C: > Users > Anandpaul > AppData > Local > Programs > Python > Python314 > Bank_transcation.py > ...

```python
import math

class BayesianFraudDetector:
    def __init__(self, prior_fraud=0.005):
        """
        prior_fraud: Base rate P(Fraud) = 0.5% (typical bank baseline)
        """
        self.prior_fraud = prior_fraud
        self.prior_legit = 1.0 - prior_fraud

        # Feature Conditional Likelihood Table: P(Feature_i = True | Class)
        # Based on historical banking statistics
        self.likelihoods = {
            "high_amount":  {"fraud": 0.70, "legit": 0.05},  # >5x avg amount
            "foreign_ip":   {"fraud": 0.60, "legit": 0.02},  # IP outside home country
            "off_hours":    {"fraud": 0.45, "legit": 0.10},  # Transaction between 1am-5am
            "failed_pin":   {"fraud": 0.35, "legit": 0.01}   # Prior failed PIN attempt
        }

    def predict_fraud_probability(self, transaction_features):
        """
        Applies Bayes' Theorem assuming naive conditional independence:
        P(Fraud | X) = [P(Fraud) * PROD(P(Xi | Fraud))] / P(X)
        """
        # Start with log-priors to ensure numerical stability
        log_prob_fraud = math.log(self.prior_fraud)
        log_prob_legit = math.log(self.prior_legit)

        for feature_name, observed in transaction_features.items():
            if feature_name in self.likelihoods:
                p_given_fraud = self.likelihoods[feature_name]["fraud"]
                p_given_legit = self.likelihoods[feature_name]["legit"]

                if observed:
                    log_prob_fraud += math.log(p_given_fraud)
                    log_prob_legit += math.log(p_given_legit)
                else:
                    log_prob_fraud += math.log(1.0 - p_given_fraud)
                    log_prob_legit += math.log(1.0 - p_given_legit)

        # Convert back from log-space using softmax normalization
        max_log = max(log_prob_fraud, log_prob_legit)
        unnorm_fraud = math.exp(log_prob_fraud - max_log)
        unnorm_legit = math.exp(log_prob_legit - max_log)

        p_fraud_given_x = unnorm_fraud / (unnorm_fraud + unnorm_legit)
        return p_fraud_given_x

    def evaluate_transaction(self, tx_id, features):
        p_fraud = self.predict_fraud_probability(features)

        # Determine action based on conditional probability thresholds
        if p_fraud >= 0.70:
            action = "BLOCK TRANSACTION & ALERT FRAUD TEAM"
            risk_level = "HIGH"
```

# Output:

```
======================================================================
       BAYESIAN CONDITIONAL PROBABILITY FRAUD DETECTION ENGINE
======================================================================
Base Rate Prior P(Fraud) : 0.5%

Transaction ID : TX_1001 (Standard Grocery Store Purchase)
  ├─ Observed Indicators : {'high_amount': False, 'foreign_ip': False, 'off_hours': False, 'failed_pin': False}
  ├─ P(Fraud | Evidence) :   0.03%
  ├─ Risk Assessment     : [LOW]
  └─ Action Required     : APPROVE (Normal Flow)

Transaction ID : TX_1002 (Late-Night Online Purchase from New Overseas IP)
  ├─ Observed Indicators : {'high_amount': True, 'foreign_ip': True, 'off_hours': True, 'failed_pin': False}
  ├─ P(Fraud | Evidence) :  86.18%
  ├─ Risk Assessment     : [HIGH]
  └─ Action Required     : BLOCK TRANSACTION & ALERT FRAUD TEAM

Transaction ID : TX_1003 (High Amount Purchase after Failed PIN Attempt)
  ├─ Observed Indicators : {'high_amount': True, 'foreign_ip': False, 'off_hours': False, 'failed_pin': True}
  ├─ P(Fraud | Evidence) :  38.05%
  ├─ Risk Assessment     : [MEDIUM]
  └─ Action Required     : CHALLENGE (Require SMS OTP / 2FA)

Transaction ID : TX_1004 (Full Fraud Vector (All Red Flags Triggered))
  ├─ Observed Indicators : {'high_amount': True, 'foreign_ip': True, 'off_hours': True, 'failed_pin': True}
  ├─ P(Fraud | Evidence) :  99.70%
  ├─ Risk Assessment     : [HIGH]
  └─ Action Required     : BLOCK TRANSACTION & ALERT FRAUD TEAM

======================================================================
```

5. **Illustrate how baise decision theory can be applied in medical diagnosis for disese prediction . give me code**

## Code:

spam_probability.py | student_performance.py | weather_forecasting.py | Bank_transcation.py | baise.py

C: > Users > Anandpaul > AppData > Local > Programs > Python > Python314 > baise.py > BayesianMedicalDecisionEngine

```python
class BayesianMedicalDecisionEngine:
    def __init__(self, prior_prevalence, sensitivity, specificity, loss_matrix):
        """
        prior_prevalence : P(D) - Baseline disease prevalence
        sensitivity      : P(+|D) - True positive rate
        specificity      : P(-|~D) - True negative rate
        loss_matrix      : Dict containing penalty points for decisions
        """
        self.p_d = prior_prevalence
        self.p_not_d = 1.0 - prior_prevalence

        # Likelihoods
        self.p_pos_given_d = sensitivity
        self.p_neg_given_d = 1.0 - sensitivity

        self.p_neg_given_not_d = specificity
        self.p_pos_given_not_d = 1.0 - specificity

        self.loss = loss_matrix

    def compute_posterior(self, test_result):
        """Calculates P(D | Test Result) using Bayes' Rule."""
        if test_result == "positive":
            likelihood_d = self.p_pos_given_d
            likelihood_not_d = self.p_pos_given_not_d
        else:  # negative
            likelihood_d = self.p_neg_given_d
            likelihood_not_d = self.p_neg_given_not_d

        numerator = likelihood_d * self.p_d
        evidence = (likelihood_d * self.p_d) + (likelihood_not_d * self.p_not_d)

        p_d_given_x = numerator / evidence
        p_not_d_given_x = 1.0 - p_d_given_x

        return p_d_given_x, p_not_d_given_x

    def evaluate_decision(self, test_result):
        p_d, p_not_d = self.compute_posterior(test_result)

        # Expected Risk R(action | X) = Sum_j [ L(action, state_j) * P(state_j | X) ]
        risk_treat = (self.loss["treat_disease"] * p_d) + \
                     (self.loss["treat_nodisease"] * p_not_d)

        risk_no_treat = (self.loss["notreat_disease"] * p_d) + \
                        (self.loss["notreat_nodisease"] * p_not_d)

        # Optimal Bayes Action minimizes risk
        optimal_action = "TREAT" if risk_treat < risk_no_treat else "DO NOT TREAT"

        return {
            "test_result": test_result,
            "p_disease": p_d,
```

## Output:

```
======================================================================
        BAYESIAN DECISION THEORY IN MEDICAL DIAGNOSIS
======================================================================
Disease Prior P(D) : 1.0%
Test Sensitivity   : 90.0%
Test Specificity   : 95.0%

--- Diagnostic Test Outcome: [POSITIVE] ---
   ├─ Posterior P(Disease | Test) :  15.38%
   ├─ Posterior P(Healthy | Test) :  84.62%
   ├─ Expected Risk of Treating   :   8.46 loss units
   ├─ Expected Risk of No Treat   :  23.08 loss units
   └─ BAYES OPTIMAL DECISION      : [TREAT]

--- Diagnostic Test Outcome: [NEGATIVE] ---
   ├─ Posterior P(Disease | Test) :   0.11%
   ├─ Posterior P(Healthy | Test) :  99.89%
   ├─ Expected Risk of Treating   :   9.99 loss units
   ├─ Expected Risk of No Treat   :   0.16 loss units
   └─ BAYES OPTIMAL DECISION      : [DO NOT TREAT]

======================================================================
```

6. **construct an Machine Learning workflow to classify hand written by applying the stages of a learning system. give me code**

## Code:

spam_probability.py | student_performance.py | weather_forecasting.py | Bank_transcation.py | baise.py | raw.py

C: > Users > Anandpaul > AppData > Local > Programs > Python > Python314 > raw.py > ...

```python
def flatten_and_normalize(matrix_8x8):
    """Flattens 8x8 matrix to 64D vector and normalizes values to [0.0, 1.0]."""
    flat = [pixel for row in matrix_8x8 for pixel in row]
    return [p / 16.0 for p in flat]

def add_handwriting_noise(vector, noise_level=0.05):
    """Simulates realistic human handwriting variations and slight noise."""
    import random
    noisy = []
    for val in vector:
        val += random.uniform(-noise_level, noise_level)
        noisy.append(max(0.0, min(1.0, val)))
    return noisy

# Generate Training Dataset (Experience E)
training_data = []
for digit_label, matrix in RAW_PATTERNS.items():
    base_vector = flatten_and_normalize(matrix)
    # Generate multiple variation samples per digit class
    for _ in range(10):
        varied_vector = add_handwriting_noise(base_vector, noise_level=0.08)
        training_data.append((varied_vector, digit_label))

# ---------------------------------------------------------------
# STAGE 2: MODEL FORMULATION (k-Nearest Neighbors)
# ---------------------------------------------------------------
class HandwrittenDigitClassifier:
    def __init__(self, k=3):
        self.k = k
        self.train_X = []
        self.train_y = []

    def fit(self, X, y):
        """Stores the training dataset in memory."""
        self.train_X = X
        self.train_y = y

    def _euclidean_distance(self, vec1, vec2):
        """Computes Euclidean distance between two 64D image vectors."""
        return math.sqrt(sum((a - b) ** 2 for a, b in zip(vec1, vec2)))

    def predict(self, test_vec):
        """Predicts digit class by majority vote among k-nearest neighbors."""
        distances = []
        for i, train_vec in enumerate(self.train_X):
            dist = self._euclidean_distance(test_vec, train_vec)
            distances.append((dist, self.train_y[i]))

        # Sort by distance and pick top K neighbors
        distances.sort(key=lambda item: item[0])
        top_k_labels = [label for _, label in distances[:self.k]]

        # Majority vote
        most_common = Counter(top_k_labels).most_common(1)
```

## Output:

```
================================================================================
================================================================
      HANDWRITTEN DIGIT CLASSIFICATION LEARNING SYSTEM
================================================================
Training Set Size : 40 samples across 4 classes
Feature Space     : 64 Continuous Pixel Inputs (8x8 Grid)
Classifier Model  : k-Nearest Neighbors (k=3)

--- Test Image A (True Label: 0) ---
      ::**++
      %%%%**%%::
      %%..  ++==
    ..##    ==++
    ::==    ++==
    ::##    ##--
    ..##::==**
      --**==
  ├─ Predicted Label : [0]
  └─ Status          : [MATCH / CORRECT]

--- Test Image B (True Label: 1) ---
        ****::
          %%==
          %%==
          ##==
          %%++
          @@==
          @@==
        ::%%@@
  ├─ Predicted Label : [1]
  └─ Status          : [MATCH / CORRECT]

--- Test Image C (True Label: 7) ---
      %%@@%%%%--
        ..::@@**
          --%%==
          ####
        ==@@::
        ****
        %%==
        **..
  ├─ Predicted Label : [7]
  └─ Status          : [MATCH / CORRECT]

--- Test Image D (True Label: 8) ---
      ==%%##--
    --%%++++%%::
    --@@..  @@..
      **%%**++
    ..%%::++@@..
    --@@    @@--
    ::@@++**@@..
      ++####::
  ├─ Predicted Label : [8]
  └─ Status          : [MATCH / CORRECT]

================================================================
PERFORMANCE EVALUATION (Measure P):
Overall Accuracy : 100.00% (4/4 correct)
================================================================
```